

Top 10 web hacking techniques of 2018



James Kettle
Director of Research
[@albinowax](#)



Published: Wednesday, 27 February 2019 at 15:45 UTC

Updated: Tuesday, 5 January 2021 at 14:10 UTC



The results are in! After an impressive 59 nominations followed by a community vote to pick 15 finalists, a panel consisting of [myself](#) and noted researchers [Nicolas Grégoire](#), [Soroush Dalili](#) and [Fildescriptor](#) have conferred, voted, and selected the 10 most innovative new techniques that we think will withstand the test of time and inspire fresh attacks for years to come.

We'll start at number 10, and count down towards the top technique of the year.

10. XS-Searching Google's bug tracker to find out vulnerable source code

This blog post by [Luan Herrera](#) looks like a straightforward vulnerability write up, right up until he innovatively uses a browser [cache timing technique](#) to eliminate network latency from a notoriously unreliable technique, making it surprisingly practical. I think we can expect to see more XS-Search bugs in the future.

9. Data Exfiltration via Formula Injection

In this [blog post](#), [Ajay](#) and [Balaji](#) explore a number of techniques for exfiltrating data from spreadsheets in Google Sheets and LibreOffice. It might be less shiny than higher-ranked items, but this is practical, easily applicable research that will be invaluable for anyone looking to quickly prove the impact of a formula injection vulnerability.

If you're wondering what malicious spreadsheets have to do with web security, check out [Comma Separated Vulnerabilities](#). It's also worth mentioning that 2018 also brought us the first documented [server-side formula injection](#).

8. Prepare(): Introducing novel Exploitation Techniques in WordPress

WordPress is such a complex beast that exploiting it is increasingly becoming a stand-alone discipline. In this presentation, [Robin Peraglie](#) shares [in-depth research](#) of WordPress' misuse of double prepared statements, with a nice touch on PHP's infamous unserialize, built on [previously shortlisted](#) research by Slavco Mihajloski.

7. Exploiting XXE with local DTD files

Attempts to exploit [blind XXE](#) often rely on loading external, attacker-hosted files and are thus sometimes thwarted by firewalls blocking outbound traffic from the vulnerable server. In a blog post described by Nicolas as 'how to innovate in a well-known field', [Arseniy Sharoglazov](#) shares a [creative technique](#) to avoid the firewall problem by using a local file instead.

Although limited to certain XML parsers and configurations, when it works this technique could easily make the difference between a DoS and a full server compromise. It also provoked a follow up comment showing an even more [flexible alternative](#).

We have built a [Web Security Academy lab](#) where you can try this technique out for yourself in a demo environment.

6. It's A PHP Unserialization Vulnerability Jim But Not As We Know It

It's been known in some circles for a while that harmless sounding file operations like `file_exists()` could be abused using PHP's `phar://` stream wrapper to trigger deserialisation and obtain RCE, but [Sam Thomas'](#) [whitepaper](#) and [presentation](#) finally dragged it out into the light for good with a robust investigation of practical concerns and numerous exploitation case studies including our friend WordPress.

5. Attacking 'Modern' Web Technologies

In which [Frans Rosen](#) shares some quality research showing that deprecated or not, you can abuse HTML5 AppCache for some [wonderful exploits](#). He also discusses some interesting postMessage attacks exploiting client-side [race conditions](#).

4. Prototype pollution attacks in NodeJS applications

It's always great to see a language-specific vulnerability that doesn't affect PHP, and this research [presented](#) by [Olivier Arteau](#) at NorthSec is no exception. It details a novel technique to get [RCE on NodeJS applications](#) by using `__proto__` based attacks that have previously only been applied to client-side applications.

I suspect you could scan for this vulnerability by adding `__proto__` as a magic word inside [Backslash Powered Scanner](#), but be warned that this may semi-permanently take down vulnerable websites.

3. Beyond XSS: Edge Side Include Injection

Continuing the theme of legacy web technologies getting a second wind as exploit vectors, [Louis Dion-Marcil](#) discovered that numerous popular reverse proxies sometimes let hackers abuse [Edge Side Includes](#) to give their [XSS](#) superpowers including [SSRF](#). This quality research demonstrates numerous high impact exploit scenarios, and also proves it's more than just an XSS escalation technique, by enabling exploitation of HTML within JSON responses.

2. Practical Web Cache Poisoning: Redefining 'Unexploitable'

This research by [er James Kettle](#) shows techniques to [poison web caches with malicious content](#) using obscure HTTP headers. I was naturally [banned](#) from voting/commenting on it, but the other panelists described it as 'excellent and extensive fresh research on an old topic', 'original and well executed research, with a very clear methodology', and 'simple yet beautiful'. I highly recommend taking a read, even if just so you can decide for yourself if I cheated my way to near-victory.

2020 update: We have released six free, interactive labs so you can [try out web cache poisoning for yourself](#) as part of our Web Security Academy.

1. Breaking Parser Logic: Take Your Path Normalization off and Pop 0days Out!

[Orange Tsai](#) has taken an attack surface many mistakenly thought was hardened beyond hope, and smashed it to pieces. His [superb presentation](#) shows how subtle flaws in path validation can be twisted with consistently severe results. The entire panel loved this research for its practicality, raw impact, and wide ranging fallout, affecting frameworks, standalone web servers, and reverse proxies alike.

This is the [second year running](#) research by Orange has topped the board, so we'll be paying close attention during 2019!

Runners up

The huge number of nominations lead to a particularly brutal community vote this year, with numerous respectable pieces of research failing to make the shortlist. As such, if the top 10 leaves you clamouring for more you might want to peruse the entire [nomination list](#) as well as [last year's top 10](#). Also, if you're wondering how to invent a technique that will land you in next year's list, I've published my personal advice on the topic in [So you want to become a web security researcher?](#).

What next?

Looking ahead to next year, I'll try to make the community vote stage a bit slicker by doing a slightly stricter filter on nominations - in particular, I'll reject vulnerability writeups that purely apply known techniques in an unoriginal way. We'll also look into improving the vote UI, and perhaps allowing comments during voting so you can explain the reasoning behind your favourite research.

This year's vote rolled around really quickly thanks to last year's occurring way behind schedule, but next year the process will be launched in January 2020. As usual, we're already [open for nominations](#). Finally, I'd like to thank everyone in the community for your research, nominations, votes and patience.

Till next year!

Update: the [Top 10 Web Hacking Techniques of 2019](#) is now out.

[top-10-techniques](#)

[Top 10 Hacking Techniques](#)

[← Back to all articles](#)

Related Research



Burp Suite

Web vulnerability scanner
Burp Suite Editions
Release Notes

Vulnerabilities

Cross-site scripting (XSS)
SQL injection
Cross-site request forgery
XML external entity injection
Directory traversal
Server-side request forgery

Customers

Organizations
Testers
Developers

Company

About
Careers
Contact
Legal
Privacy Notice
Modern Slavery Statement

Insights

Web Security Academy
Blog
Research
Engineering



[Follow us](#)

© 2026 PortSwigger Ltd.

